TP 2 : Hardware Software Interface (HSI)

Florent Bednarek - INEM 2025/2026
florent.bednarek@gmail.com

Partie 2 : Arithmétique & Assembleur

Formule de la luminance :

$$Gray = 0.299 * R + 0.587 * G + 0.114 * B$$

1. Écrivez une fonction `void to_grayscale_float(Image *img)` qui applique la formule ci-dessus aux pixels d'une image.
    - Utilisez des variables `float` pour le calcul.
    - Castez le résultat final en `uint8_t` pour le stocker (R=G=B=Gray).
2. Encadrez l'appel à la fonction avec `clock_gettime(CLOCK_MONOTONIC)` et exécutez la conversion dans une boucle de **100 itérations** pour stabiliser la mesure du temps moyen.

Sur un processeur embarqué, sans unité de calcul flottant (FPU), les calculs float sont émulés logiquement, c'est lent et coûteux. Nous allons donc passer en **Virgule fixe**.

3. Créez une fonction `void to_grayscale_fixed(Image *img)`
4. Établissez un nouveau calcul de la luminance qui s'affranchit de l'usage de float en utilisant uniquement des entiers (uint32_t) et des opérations binaires (+, *, >>).
5. Mesurez le temps d'exécution sur **100 itérations** et comparez avec la version Float.

Nous allons maintenant comparer les 2 solutions en analysant le code assembleur. vous pouvez le générer avec la commande suivant :

```
gcc -S -masm=intel -O0 main.c -o main.s
```

1. Comparez les fonctions `to_grayscale_float` et `to_grayscale_fixed`. quels types d'instructions repérez- vous ?
2. Cross-Compilez votre programme avec une cible ARM Cortex-M0 sans FPU avec les options suivantes.

```
arm-none-eabi-gcc -mcpu=cortex-m0 -msoft-float -S
```

3. Quelles fonctions particulières apparaissent dans la version float ?
4. **(Optionnel)** Vous pouvez tester les temps d'exécution sur Cortex-M0 avec l'émulateur QEMU. Instructions en annexe 1

Partie 3 : Caches, Layouts Mémoire & SIMD

Actuellement, votre mémoire ressemble à ça (Array of Structures - AOS) :

`[RGB] [RGB] [RGB] [RGB] ...`

Si le CPU charge une ligne de cache (64 octets), il charge R, G et B. Mais pour certains algos, on veut, par exemple, ne pouvoir traiter que le canal Vert.

Nous allons passer en Structure of Arrays (SOA) (ou "Planar") :

`[RRRRRR...]` suivi de `[GGGGGG...]` suivi de `[BBBBBB...]`

1. Définissez une nouvelle structure `ImageSOA` :

```
typedef struct {
        int width, height;
        uint8_t *r;
        uint8_t *g;
        uint8_t *b;
} ImageSOA;
```

2. Écrivez une fonction pour convertir l'image dans ce nouveau format `ImageSOA* convert_to_soa(const Image *img)`
    - *Astuce* : Essayez d'allouer un seul gros bloc `malloc(w * h * 3)` et faites pointer `r, g, b` aux bons endroits (offsets) pour éviter la fragmentation.
3. Quelle est la taille de la structure en partie 1 ? en partie 3 ? Commenter.
4. Écrivez les 2 fonctions suivantes pour appliquer un filtre d'amplification sur le canal Vert.
    - `void process_green_aos(Image *img)`
    - `void process_green_soa(ImageSOA *img)`
    - Amplification canal vert = `max(G * 1.2, 256)`

5. Analysez l'utilisation de la mémoire pour ces 2 solutions a l'aide de l'outil **Cachgrind.** Analysez le taux de L1 Data Cache Misses (`D1mr` / `D1mw`) pour la version AoS et la version SoA. Expliquez pourquoi la version SoA tire un meilleur parti des lignes de cache (Cache Line de 64 octets)

```
valgrind --tool=cachegrind ./programme
```

La disposition SoA rend le tableau `g` contigu en mémoire, ce qui permet au compilateur de générer facilement des instructions vectorielles (SSE/AVX sur x86 ou NEON sur ARM).

6. Compilez votre programme selon 3 configurations distinctes :

    - **Sans optimisation :** `gcc -O0 main.c -o prog_O0`
    - **Optimisation scalaire :** `gcc -O2 -fno-tree-vectorize main.c -o prog_O2_nosimd`
    - **Vectorisation maximale :** `gcc -O3 -march=native main.c -o prog_O3_simd`

7. Mesurez les temps d'exécution sur le traitement mono-canal pour chaque binaire.
8. Quel gain de performance observez-vous entre la version `-O2 -fno-tree-vectorize` et `-O3 -march=native` sur la structure SoA ?

Annexe 1 : Utilisation de QEMU

Prérequis (Linux / Ubuntu / Debian)

1. Ouvrez un terminal et installez la chaîne de compilation ARM et QEMU :

```
sudo apt update
sudo apt install -y qemu-system-arm gcc-arm-none-eabi
gdb-multiarch
```

2. Ecrivez une fonction qui permet de print des caractères en UART

```
#define UART0_DR *((volatile unsigned int *)0x4000C000)

void print_uart(const char *s) {
    while (*s) {
        UART0_DR = *s++;
    }
}
```

3. Créez la table des vecteurs et le script de démarrage : startup.c.

Le processeur Cortex-M nécessite une table de vecteurs d'interruption au démarrage pour initialiser le pointeur de pile (SP) et l'adresse d'exécution.

`startup.c` :

```

extern void main(void);
extern unsigned long _estack;

// Table des vecteurs minimale
__attribute__ ((section(".isr_vector")))
unsigned long *vector_table[] = {
    (unsigned long *)&_estack, // Initial SP
    (unsigned long *)main      // Reset Handler
};
```

4. Créez un fichier linker.ld pour indiquer l'organisation de la mémoire RAM et Flash.

`linker.ld` :

```
MEMORY
{
  FLASH (rx) : ORIGIN = 0x00000000, LENGTH = 256K
  RAM (rwx)  : ORIGIN = 0x20000000, LENGTH = 64K
}

_estack = ORIGIN(RAM) + LENGTH(RAM);

SECTIONS
{
```

```
  .text : {
    *(.isr_vector)
    *(.text*)
    *(.rodata*)
  } > FLASH

  .data : {
    *(.data*)
  } > RAM AT > FLASH

  .bss : {
    *(.bss*)
  } > RAM
}
```

5. Compilez et générez le fichier ELF binaire en ciblant l'architecture Cortex-M0 (cortex-m0).

```
arm-none-eabi-gcc -mcpu=cortex-m0 -mthumb -T linker.ld startup.c
main.c -o firmware.elf -nostartfiles
```

6. Lancez le programme dans QEMU

Exécute le fichier binaire sur la machine virtuelle lm3s6965evb (carte basée sur Cortex-M0) en redirigeant le port série vers le terminal :

```
qemu-system-arm -M lm3s6965evb -cpu cortex-m0 -kernel firmware.elf
-nographic
```

Pour aller plus loin : Débogage avec GDB

Lancer QEMU en mode pause (`-S`) sur le port 1234 (`-gdb tcp::1234`) :

```
qemu-system-arm -M lm3s6965evb -cpu cortex-m0 -kernel firmware.elf
-nographic -gdb tcp::1234 -S
```

Dans un autre terminal, ouvrir GDB et se connecter :

```
gdb-multiarch firmware.elf -ex "target remote localhost:1234"
```